Project Specification

Rayan Belahcene
Lou Reina–Kuntziger
Seerat Yasir

2026-11-10

1 Introduction

Computer systems rely on the assumption that stored data remains unchanged unless software explicitly modifies it. However, transient hardware faults or fault injection attacks can cause a bit to change from 0 to 1 or vice versa. These events are called bit flips and can corrupt data or instructions, potentially causing incorrect results, privilege escalations, application crashes, or system failures. Some errors remain unnoticed because the affected application continues executing despite using corrupted values also known as data corruption.

In systems that are more vulnerable to bitflip errors due to their environments, such as satellites, bitflips can occur often enough to be concerning for the execution of code.

2 Possible vulnerabilities

- Bitflipping a permission bit can lead to privilege escalation.
- Bitflipping an access key can break a connection.
- Bitflipping executable memory can cause instruction skips.
- Bitflipping pointers (e.g. program counter), return addresses and flags can cause instruction jumps or skips, leading to control flow hijacking. This can in turn lead to unauthorized access or running of arbitrary code.
- Bitflipping in memory can cause data corruption or in worst cause, corrupt boot and crash system.
- Bitflipping an instruction or register can change it to another, making the program do something different.

3 Project scope and requirements

Our goal is to check the integrity of data we use, meaning that it has not changed from the last legitimate operation. This includes application data, instructions, and critical registers such as PMP.

The project will exclude CPU execution faults and physical PMP-register faults.

The scope is reduced to prevent only process memory corruption.

3.1 Tasks

- Create a monitor process.
- Create a fake monitored process.
- Create a fault injector for testing.
- Compute hashes or checksums of the process memory.

- Store redundant hashes.
- Compare hash between each instruction.
- Create an ECC (error correcting code) ?
- Correct bit flips through ECC.

3.2 Optional requirements

- Add instruction skip prevention through CFG hashes.
- Find a way to reduce the computation and memory overhead.

References

Course in Cybersecurity Overview, DD2391.